



New Perspectives on Cartesian Genetic Programming: A Survey

Mark Kocherovsky¹✉ , Henning Cui² , Illya Bakurov¹ ,
Michael Heider² , Roman Kalkreuth³ , and Wolfgang Banzhaf¹

¹ Department of Computer Science and BEACON Center for the Study of Evolution in Action, Michigan State University, East Lansing, MI 48824, USA

kocherov@msu.edu

² Organic Computing Group, University of Augsburg, 86159 Augsburg, Germany

michael.heider@uni-a.de

³ Chair for AI Methodology (AIM), Faculty of Computer Science, RWTH Aachen University, 52062 Aachen, Germany

kalkreuth@aim.rwth-aachen.de

<https://banzhaf-lab.github.io/>, <https://aim.rwth-aachen.de/>

Abstract. Over the past twenty-five years, certain practices and assumptions in Cartesian Genetic Programming (CGP) have become conventional wisdom, yet recent research challenges their validity. This position paper critically examines these long-standing beliefs and proposes evidence-based alternatives for the CGP community. We address four misconceptions: The purported ineffectiveness of crossover operators; the overlooked impact of positional bias; problems with tournament selection on Boolean benchmarks; and the limitations of single-domain analysis. Through a review of recent literature, we identify a key principle underlying successful CGP operators—the preservation of node structural integrity during genetic operations. We discuss current best practices including rigorous hyperparameter tuning, cross-domain benchmarking, node-preserving operators, and modern fitness function design. Our analysis reveals that many accepted CGP practices, including the ubiquitous $(1 + 4)$ evolution strategy, lack generalizability across problem domains. We believe that by reconsidering these assumptions and adopting the recommendations presented here, researchers and practitioners can develop more effective and robust CGP implementations.

Keywords: Cartesian Genetic Programming · Crossover · Positional Bias · Node Preservation · Best Practices · Future Perspectives

1 Introduction

Cartesian Genetic Programming (CGP) is a branch of Genetic Programming (GP) introduced by Julian Miller and his collaborators in the late 1990s [62, 64]. The basic idea behind CGP is that a program is represented as a direct acyclic graph (DAG) consisting of a series of nodes, each taking inputs, performing

an operation, and providing an output. Since then, there has been a wealth of investigation into CGP, its various dynamics, and towards the development of more effective operators.

Over the past twenty-five years, certain assumptions about CGP have become the default mode of operation for researchers in the field. These include the assumption that crossover is generally deleterious in CGP search, the lack of mitigation for positional bias, and the CGP benchmarking methodology. This work aims to discuss these assumptions holistically in order to make recommendations for potential best practices for CGP implementations. We base our advice on the last twenty-five years of work in and around CGP. With this contribution, we also want to propose avenues for future work.

Our position covers four related topics. After a brief introduction to CGP (Sect. 2), we first cover misconceptions in the older literature that are beginning to be unveiled by newer work in Sect. 3. Second, in Sect. 4 we holistically cover operators shown to be well-performing in the recent literature. In Sect. 5, we discuss best practices in CGP for users regarding traversing the search space and establishing strong generalized operators. Finally, in Sect. 6, we discuss new directions of research based on the overview given in the other sections.

2 Brief Overview of Cartesian Genetic Programming

CGP individuals are feed-forward directed acyclic graphs, and thus consist of nodes and edges. A node is either an input node, which takes in data samples; an output node, which redirects the output of previous nodes without any additional modification and forms part of the program output; or a function node, which performs calculation(s) on inputs and returns an output. It is important to note that the input for function nodes may be from any node closer to the start of the genome, meaning that function node at position m , can take input from any input node i or a subset of function nodes at positions $[0, m)$. The edges of the graph therefore carry values from one node to another for operations or output.

CGP individuals are normally represented by a one-dimensional string of values. A CGP node has to have a given number of possible operator inputs called an *arity*. An arity of two ($a = 2$) means that the operations will be either unary or binary. A CGP individual p can be written as

$$p = i_0|i_1|\dots|i_\alpha|g_0|a_{01}|a_{02}|g_1|a_{11}|a_{12}|\dots|g_\beta|a_{\beta 1}|a_{\beta 2}|o_0|o_1\dots o_\gamma \quad (1)$$

where, given α -dimensional data, at most β function nodes, and a target output-dimensionality of γ , we will thus far have α genes representing input nodes, $\beta \cdot a$ genes for function nodes (with an operator gene g_m and a genes indicating which nodes to use as input), and γ genes representing which indices to return as output. Each gene is separated by the `|` character to make it easier to read. Input nodes are excluded from genomic operations since their value is a given data sample rather than dependent on other nodes' calculations [54].

Simple point mutation, for example, can be executed by picking a gene at random and exchanging it for another legal value. One-point crossover can be

implemented by choosing an index at random, and performing recombination as in any other genetic algorithm. Typically, a $(1+\lambda)$ evolution strategy (ES) is used in the evolutionary loop, as it is generally thought that basic crossover methods are not effective for CGP [13], although we discuss this as a misconception in Sect. 3.1.

3 Misconceptions in the CGP Community

Over the years, standard practices to CGP’s usage have been established. However, they are also oftentimes adopted without question in theoretical studies or use-case specific applications, even though they might impair CGP’s evolutionary processes. In this section, we want to draw attention to selected major misconceptions in CGP to aid researchers and practitioners in the future.

3.1 Effects of Crossover

Early experiments in CGP operators leaned towards standard genetic algorithmic operators rather than specifically GP-oriented operators. Clegg et al. [13] from 2007 is often cited as the original source that stated the CGP crossover problem, in which crossover is generally regarded as a universally-deleterious operator in CGP. In the paper, the authors implement a new crossover based on real-valued crossovers found in the genetic algorithm literature [1] and describe it as more effective than a crossover operator matching a one-point crossover from standard GAs where the nodes are kept intact (more about this kind of method will be described in Sect. 5.2). However, Turner [83] and Kocherovsky et al. [54] found that the real-valued operator was not effective compared to other crossover operators.

Recent results by Kalkreuth [44, 45, 47], Kocherovsky et al. [52], and Cui et al. [14] demonstrate an alternate possibility. In these works, crossover operators are not generally deleterious, and can even improve performance compared to the canonical $(1+\lambda)$ method. We thus postulate that the negative attitude towards crossover in CGP may be misplaced.

3.2 Tournament Selection Negatively Affects CGP on Boolean Benchmarks

According to Cui et al.’s [14] findings, *standard tournament selection* greatly worsens the performance of CGP on *Boolean benchmarks*¹. For example, on all tested Boolean benchmarks, all CGP versions using tournament selection in a (μ, λ) -ES with elitism had worse fitness values than CGP with random selection in a $(\mu + \lambda)$ -ES. This outcome was independent of the CGP version and crossover used. It is also important to note that such poor performances occurred even after an extensive hyperparameter-tuning process.

¹ Other benchmark types, e.g. symbolic regression, were not affected.

A possible reason for this aforementioned problem might be due to a specific characteristic of Boolean benchmarks: They have a *deceptive fitness landscape* [86]. Due to the Boolean nature of this benchmark type, various different solutions lead to the same fitness value and numerous solutions are also treated as equally good. Thus, CGP with tournament selection might not be suitable for this specific kind of benchmark. This might be due to the properties of this selection method: Blickle and Thiele [10] proved that tournament selection has the lowest loss of population diversity. This is paired with the highest expected variance of the fitness distribution of the population. The combination of both characteristics might lead to CGP not being able to properly explore the search space (Fitness landscapes are discussed more in-depth in Sect. 6.4).

To the best of our knowledge, this finding has not been mentioned yet by other publications, and might have substantial consequences for the CGP research community. Some foundational work (e.g. [42, 62], or [88]) used CGP with a tournament selection. *If* other researchers are able to replicate and validate this outcome, parts of the literature’s empirical work and conclusions should then be subject to revision.

3.3 Positional Bias

CGP enforces a feed-forward grid. This leads to *positional bias* (also called *length bias*), as was found by Goldman and Punch [30]. This bias describes a non-uniform distribution of active nodes in an individual. Computational nodes near input nodes have a *higher chance* of being active compared to the nodes near output nodes. This is due to the feed-forward nature of CGP. Function nodes near input nodes can be used by the majority of other nodes in the CGP graph. However, function nodes near the output nodes have far fewer opportunities to be used. As a result, there are also less nodes that could mutate their connections to nodes on the output side, decreasing their probability to become active.

Positional bias negatively impacts CGP’s evolutionary search process because it increases the difficulty of solving certain tasks or evolving specific structures, leading to decreased performance [31, 73]. To counteract positional bias, Goldman and Punch [30, 32] and Cui et al. [15, 17] proposed extensions to CGP. By *reordering active nodes*, this issue can be mitigated and fitness improved.

3.4 Cross-Domain Analysis

In the past, two problem domains in particular have been used to better understand the search properties of CGP. These are symbolic regression (SR) and logic synthesis (LS), the latter of which can be considered a significant pillar in the application domain since the early days of CGP. Various studies focusing on hyperparameter parameterization optimization of CGP revealed significant differences when benchmarks from SR or LS were used with CGP. In the framework of the first study, Kaufmann and Kalkreuth [49] tuned essential parameters of a $(1 + \lambda)$ -ES, such as offspring population size (λ), genome length, and mutation rate by using a SOTA hyperparameter optimizer called *irace* [58]. The

outcome of this first study indicated a rather contradictory situation between settings that work effectively in SR and LS. For SR, it was found that large populations can be effectively used, while in LS configurations with minimal populations performed best. Moreover, the study clearly demonstrated that the $(1 + 4)$ -ES cannot be considered a general effective choice for configuring CGP, which had been postulated by Miller [63] based on his experiments with Boolean functions.

Later work by Kalkreuth [45] further pursued this topic by including recombination operators such as subgraph crossover and block crossover [42]. Mainly, the study confirmed the results reported in [49], but it also demonstrated that recombination-based algorithms can work to some extent in SR. In the LS domain, on the other hand, it was found that a mutation-only $(1 + \lambda)$ -ES performed best for the Boolean functions considered. The effectiveness of the minimal setting of the λ parameter was later confirmed on a larger scale by Kalkreuth et al. [48] who introduced a benchmark suite for logic synthesis with respective baseline results obtained with CGP.

In [45], it was highlighted that SR and LS as tackled within the GP paradigm markedly differ by the nature of the fitness functions. Practitioners typically use a continuous fitness function, such as mean squared error for symbolic regression problems, but in logical synthesis problems, the Hamming distance is used predominantly. Fitness functions are described further in Sect. 5.4.

Overall, cross-domain studies in CGP broke with the dogma that the $(1 + 4)$ -ES can be considered a generally effective choice since there is more evidence that mutation-only algorithms with a very small population size perform quite well in LS, but the success and effectiveness of algorithms that use larger populations and recombination has been demonstrated in SR to some extent.

4 Successful Genetic Operators in CGP - A Holistic View

A holistic analysis of successful crossover and mutation strategies in CGP reveals a consistent pattern: the structural integrity of *nodes*—as self-contained computational units—must be preserved to ensure effective variation. This conclusion stems from two major contributions.

First, Kalkreuth [44] demonstrated the benefits of explicitly constraining one-point crossover to act only on active nodes of CGP’s DAG, thus performing recombination of active subgraphs rather than arbitrary genome fragments (cf. Sect. 4.1). In this way, intra-node dependencies were kept intact and crossover’s destructiveness was reduced.

Second, more recently, Kocherovsky et al. [52] generalized the principles behind subgraph crossover beyond one-point crossover. Specifically, it was shown that explicitly preserving node boundaries—regardless of whether the nodes are active or inactive—substantially improves the likelihood of producing fitter offspring. This method resulted in classical one-point and uniform crossover operators achieving performance close to that of subgraph crossover, and in some symbolic regression benchmarks, even outperforming it. Beyond crossover, node

boundaries preservation was also extended to mutation. Kocherovsky et al. found that using node-level mutation (replacing the entire instruction), instead of traditional gene-level mutation (modifying a single gene within a node), consistently led to better search outcomes [52]. Moreover, as proposed by Kalkreuth [45], these findings allow to question the strict reliance on phenotypically-active substructures. In fact, enforcing node preservation regardless of the node’s activity status may offer additional benefits by promoting neutrality, which was found to be advantageous for evolutionary search due to its role in enabling a broad exploration and contributing to simplicity bias [5–7, 38–40]. These aspects, however, still need to be investigated further.

These results point to a general principle: node-preserving operators in CGP contribute to the evolvability. Moreover, we argue that treating nodes as atomic units of computation can facilitate the modularity, emergence, retention, and reuse of useful building blocks, ultimately leading to more robust and quicker evolutionary search. In the following subsections we discuss subgraph crossover and node connectivity in more detail.

4.1 Subgraph Crossover

The subgraph crossover technique for CGP draws inspiration from the subtree crossover used in tree-based GP. This method performs recombination while taking the CGP phenotype into careful consideration. Simply exchanging parts of the genotype, as noted by Clegg et al. [13], can lead to poor results.

In CGP, an individual’s phenotype is defined by its active graph trace identified during evaluation, which ultimately determines its semantic output. Therefore, subgraph crossover focuses solely on recombining the genetic material found within these active paths. This approach aims to minimize the disruption typically caused by traditional genotypic single-point crossover in CGP, offering a more meaningful recombination of subgraphs.

To describe the procedure of subgraph crossover, let n_i represent the predefined number of input nodes and n_f the predefined number of function nodes. In typical CGP representations, inputs are indexed from 0 to $n_i - 1$ and the function nodes of each graph are indexed from n_i to $n_i + n_f - 1$. The nodes which lie between the input and output nodes are denoted as function nodes. Crossover is performed with two pre-selected parents that are denoted as P_1 and P_2 . For the crossover procedure, the number of nodes of the active function nodes is necessary. The node numbers of the active nodes of P_1 and P_2 are stored in two lists M_1 and M_2 . The active nodes are determined by a backward search in the evaluation procedure. To define one suitable crossover point, we define two possible crossover points C_{P_1} and C_{P_2} of the two parents. Given this information on active nodes and the length of a path, we can choose two possible crossover points. The possible crossover points C_{P_1} and C_{P_2} are chosen by chance in the range of the active function nodes which are stored in M_1 and M_2 . Input or output nodes are not allowed as possible crossover points. A general crossover point CP is defined by choosing the smaller crossover point from C_{P_1} and C_{P_2} . The reason for this is that the subgraphs of the parents, which will be

placed in front of or behind the crossover point of an offspring’s genome should be balanced. The representation of CGP allows active paths of an individual, possibly starting in the middle or back of the graph. The subgraph which will be placed in front of the crossover point has to start at more leading active nodes.

4.2 Node Connectivity

In subgraph crossover, covered in Sect. 4.1, it is required to perform crossover in such a way that individual instructions remain intact until *after* recombination, at which point a node can be rewired for continuity [44]. In a similar vein, Kocherovsky et al. [53] postulated that, whereas linear genetic programming makes use of memory external to the program as a way to anchor substructures, CGP has no such method to prevent destructive operations, and thus when a gene is altered, a CGP program is more likely to suffer catastrophic effects on the graph trace. Based on their experimental results, [54] suggested that “traditional” methods of crossover failed to take into account that instructions are *interdependent* because instructions take other instructions as inputs. The authors also remarked that nodes adjacent to each other in the genotype are functionally related, as each node can take input from any other node towards the front of the genome.

In 2025, Kocherovsky et al. demonstrated that Kalkreuth’s subgraph operator dramatically improves crossover operations [52]. Subgraph crossover explicitly takes into account the structure of the DAG; it includes a step where the first active node immediately after the crossover point has its connections rewired to ensure that both subgraphs remain active. The attention to node connectivity is likely one source of the literature’s observations of subgraph being effective.

Following these works, we stress that *properly understanding and accounting for the nuances of node connectivity is vital for developing more effective crossover operators in CGP*. Without taking into account these nuances, it is less likely that a beneficial pattern will emerge. This limits the ability of evolving programs to exploit beneficial substructures by opening these substructures to large amounts of destructive forces. Making sure node integrity is maintained decreases the chances of a substructure being destroyed by an errant crossover or mutation in the wrong place.

5 CGP Best Practices

In the following sections, we highlight some best practices which we deem vital for using CGP and advancing its state-of-the-art productively.

5.1 Hyperparameter Tuning and Experimental Design

The Evolutionary Computation community agrees on the importance of good experimental design—which also includes *tuning hyperparameters* if necessary. However, most CGP related researchers still use an elitist $(\mu + \lambda)$ -ES with $\mu = 1$ and $\lambda = 4$. While this does indeed create some comparability to previous works, it may also lead to major problems. For example, in the context of Boolean and symbolic regression benchmarks, Kaufmann and Kalkreuth [49, 50] and Trautwein et al. [82] showed that the $(1 + 4)$ -ES is seldom the best choice. In almost all benchmarks, different combinations of μ and λ lead to better performance values. Without tuning hyperparameters, experimental results could lead to fallacies and wrong conclusions.

Another characteristic of good experimental design is a *broad spectrum of up-to-date benchmarks*. But many research papers still use out-dated and/or too simple benchmarks—even though they might be disapproved by the GP community [61, 87]. By using state-of-the-art benchmarks or benchmark suites [28, 48, 72], this problem can easily be eliminated.

Both problems also play a vital role in *crossover's negative reputation in CGP*. Clegg et al.'s [13] work is often cited as an argument against the use of standard crossover techniques. However, given today's standards, the authors' experimental design could be called into question. They only based their results on a single, simple quadratic function without any hyperparameter tuning. Both aspects might greatly impair CGP performance, and their conclusion of crossover's negative influence on CGP should be faced with skepticism from a modern perspective.

Lastly, good benchmarking practices and experimental design also include testing the same configuration multiple times with independent random seeds. In some cases, a train-test split is also necessary.

We also want to highlight the importance of (state-of-the-art) statistical analysis [11, 56]. Oftentimes, using a Student's t-test (or its variants) might lead to faulty conclusions as wrong statistical prerequisites are assumed. Recently, Cui et al. [14–17] or Kocherovsky [52] et al. used the Plackett-Luce model [11] for operator comparisons. In short, the model uses the rankings of found models across operators to determine the probability that a given operator will produce a model better than the other tested operators. This probability can show differences in performance that are perhaps not obvious from looking at the distribution or quartiles alone. In addition, it doubles as a relative significance measurement. For a more in-depth guide on best practices, we refer to Bartz-Beielstein et al. [8].

5.2 Enforcing Node Integrity

In Sect. 4.2, assumptions on the functional relationships of nodes were discussed, where it was put forward that Subgraph Crossover owes its effectivity in part to explicitly taking into account the structure of the DAG. Another reason, as put forth in Kocherovsky et al. [52], is that the subgraph crossover *requires function*

nodes to remain intact throughout the operation. In other words, a crossover point could not be chosen that would separate $a_{n1}a_{n2}g_n$ (see Eq. (1))—*as already standard in tree-based GP and LGP*. The authors tested generalizing this strategy to ten symbolic regression problems using n-point and uniform crossover and found that forcing node integrity improves the evolutionary algorithm’s ability to search in most cases. The authors called this “enforcing node integrity” and “node preservation” since the node is preserved throughout. We want to point out that a similar method was presented by Clegg et al. in 2007 [13] for one-point crossover, but as covered in Sect. 5.1, there is reason to be skeptical of their methodology.

The authors of [52] also found that the same effect occurs when the same restriction is applied to mutation. Instead of traditional point mutation, a “macro” mutation was performed by replacing whole nodes with new randomly-generated operations. Even comparing canonical $(1 + \lambda)$ CGP with the different mutation operators shows that explicit node preservation during mutation also improves the search, and forcing the restriction on both crossover and mutation together provides even more benefits.

Despite these results being very new, we can comfortably recommend enforcing node integrity as an effective practice for CGP users. We also highly recommend further investigation into the topic to measuring the different degrees of phenotypic destruction brought about by node preservation compared to a more typical operator.

5.3 Cross-Domain Benchmarking

Based on the results of the hyperparameter studies surveyed in Sect. 3.1, we emphasize that cross-domain benchmarking can lead to a more holistic perspective on the success of recombination-based or mutation-only algorithms in CGP. We stress that merely focusing on one problem already led to premature generalization of dogmas such as the assumption that the use of $(1 + 4)$ -ES is a good choice in general. We therefore recommend evaluating new methodologies on at least two SOTA benchmarks that differ remarkably in terms of the optimization objective and application scope. In the continuous domain, we suggest SRBench [72], while in the discrete domain General Boolean Function Benchmark Suite (GBFS) [48], and General Program Synthesis Benchmark Suite I and II (PSB) [36, 37] can be considered adequate choices. For classification problems, Penn Machine Learning Benchmark (PMLB) [71] is a contemporary choice.

5.4 Fitness Functions and Evaluation Strategies

The CGP literature features various fitness functions depending on the type of problems used for benchmarking. As mentioned in Sect. 3.4, logical synthesis benchmarks typically use the Hamming distance as measurement [44] or an alternative metric such as proportion of correct mappings [14, 15, 17]. In symbolic regression benchmarks, a continuous function is used. For example, the sum of absolute differences (SAD) [44–46], mean absolute error [14, 15], and root mean

squared error (RMSE) [34]. Clegg et al. used sum of square error in 2007 [13]. In addition, recent literature includes *iterations to solution*; how many fitness evaluations are necessary until a solution is found. Note that while in logical synthesis an exact solution is expected, a solution in symbolic regression problems can be defined as a fitness under a given threshold.

In 2023, Haut et al. [34] found that using the Pearson Correlation r in the fitness function tends to significantly improve search. Specifically, using $f(x) = 1 - r^2$ found models that, had much smaller RMSE values than those models found using RMSE itself. This is because the correlation function takes into account the global behavior of models to match its overall shape to the given data, in contrast to local error functions that only take into account the distance between individual points at a given x value. Correlation has been used in lieu of RMSE in several recent papers [3, 4, 52–54]. In the Boolean domain it was found that fitness functions that effectively translate the symmetry of solutions into neutrality can lead to more efficient fitness functions [40].

6 New Directions for Research

After the introduction of best practices, as well as common mistakes, we now want to highlight future research directions we believe are of high relevance.

6.1 Foundational Research

Over the last several years, new ideas have emerged and have been developed in the literature. The idea of standard crossover operators having a negative influence on CGP was challenged by some researchers as covered in Sects. 3.1, 4.1, and 4.2. Furthermore, CGP using a standard tournament selection (with or without elitism) as selection/replacement strategy performs exceptionally bad on Boolean benchmark compared to CGP using its standard $(1 + 4)$ -ES, at least with the traditional Hamming distance fitness function (cf. Sect. 3.2).

Based on the previously mentioned findings, we want to draw attention to more CGP related foundational research:

1. Why does tournament selection worsen CGP’s performance on Boolean benchmarks?
2. In general, how do different evolutionary operators and their combination affect CGP?
3. Can the positive effects of standard crossover operators (1-point, 2-point, uniform crossover, ...) be validated by other researchers?

6.2 Analyzing Search Spaces at the Semantic Level

Based on the result of various cross-domain benchmarking that has been surveyed in Sect. 5, a step that should be considered in future work would be the analysis of the respective search spaces and the search behavior of the algorithms

tested to ultimately better understand what constitutes the success of a CGP algorithm in the respective problem domain. Initial work in this direction has been presented by Kalkreuth [45] by obtaining the distribution of the semantic spaces in SR and LS that could explain the difference in the search behavior between two problem domains. However, this has been merely done on a case-by-case basis by considering a relatively small set of problems. Therefore, we think that a more quantitative study is missing in the field that allows more general conclusions.

6.3 Homologous Methods in CGP

Molecular biologists have long noted that in real-world reproductive processes, genetic recombination tends to occur where genomes are rather similar to one another. This phenomenon is termed *Homology*, and is thought to emerge from base-pair bonding mechanisms [29, 41]. Hence, crossover operators inspired by homologous operations have been developed for genetic algorithms [43, 65] but have received particular attention in the genetic programming space.

While some early work was considered part of schema theory [76], Francone et al. [29] introduced a method which they call “sticky crossover”, which swaps instructions in linear genetic programs by forcing the crossover point to be at the same index in each program, arguing that homology—i.e. similarity between instructions at a given index—will emerge on its own. They reported that sticky crossover improved fitness and reduced bloat, but also reported that the emergence of homology relies on sticky crossover making up an overwhelming majority of crossover operations. A few years later, Platel et al. [74] introduced what they called “maximum homologous crossover” for LGP, which computes the best alignment for nodes. “Alignment” in this case refers to using string edit distance metrics to transform program x into program y using the minimal amount of operations.

However, to our knowledge homologous methods have not been consciously introduced into CGP until 2023. Torabi et al. [81] implemented a homologous crossover with CGP because they were specifically assembling convolutional neural networks (CNNs), and thus represent instructions as strings, where each character maps to a CNN component. From a technical perspective, one-to-one mapping enabled the authors to explicitly take homology into account. Therefore, in-line with our recommendations on accounting for node connectivity and enforcing node integrity (Sects. 4.2 and 5.2), we recommend introducing homologous crossovers to CGP by representing instructions as whole units rather than a collection of disparate genes.

6.4 Fitness Landscape in GP and Search Trajectories

In GP, the fitness landscape (FL) defines the mapping between each candidate program and its corresponding fitness value. The geometry of this landscape governs how evolution navigates towards optimality [7].

In continuous optimization the FL can be understood as measurable fitness gradients that can be visualized and analyzed. In two dimensions, for example, the FL can be visualized as a 3D surface. Smooth, convex landscapes typically lead to efficient optimization, whereas rugged landscapes with multiple local optima make convergence more difficult [59,60]. However, when a search space is a high-dimensional mix between mostly discrete terms and real-valued constants, and there is no a consistent way to measure *distance* between solutions that reflects both structure and semantics [85], the FL becomes much more difficult to understand. Small structural mutations may produce large and unpredictable behavioral changes, making it difficult to define meaningful notions of distance, continuity, or neighborhood. Moreover, the mapping between genotype, phenotype, and behavior (the program’s output) introduces neutrality, further complicating visualization and interpretation [5,6,20].

To better understand the behavior of metaheuristics, Search Trajectory Networks (STNs) have been proposed as data-driven tools to visualize and analyze how evolutionary algorithms explore FLs [67,68]. In STNs, nodes represent distinct search states (e.g., genotypes, phenotypes, or behaviors), and edges represent transitions between them across generations. By capturing evolutionary trajectories, neutral drifts, and stepping-stone transitions, STNs reveal patterns of exploration and exploitation and the connectivity of promising regions. Initially, this tool was proposed in the context of continuous problem solving, but later it was applied on Boolean functions with linear GP [40], navigation tasks with tree GP [66], and symbolic regression with CGP [20]. In particular, De La Torre et al. [20] found that both the genotype and the phenotype space suffer from a high degree of structural and semantic redundancy on symbolic regression tasks, making it difficult to interpret search trajectories of CGP. To provide a coarser and more understandable look into population dynamics, they proposed using STNs on the behavioral level, which dramatically reduced the number of unique locations, revealing to which extent different configurations and runs genuinely explored the search space.

By bridging FL understanding and STNs, it becomes possible to investigate why and how different hyper-parameters and/or evolutionary operators succeed, fail, or interact. This allows advancing our understanding of GP’s search dynamics beyond simple population-level statistics. Despite the existence of tools like STNs, FL analysis in GP and CGP in particular is still poorly covered in the literature, although it can reveal very important features about an algorithm’s suitability for solving a given problem.

6.5 Self-adaptive CGP

The effects of hyperparameterisation on evolutionary algorithms have been studied since the early days of the field [19,23]. A more direct approach consists of sampling a range of different hyperparameters for a given problem using cross-validation (aka hyperparameter tuning), and then using the optimal configuration to solve it. At time of writing, this is the established best practice as described in Sect. 5.1, but it carries several drawbacks:

1. The test values are typically predefined, therefore a limited range of hyperparameters is explored and the optimal choice can be eventually overlooked;
2. A significant amount of computational resources must be allocated during the testing phase to befittingly estimate the optimal hyperparameters;
3. Since the hyperparameters often interact in a complex manner, if the values are tuned one at a time, one may end with a suboptimal choice. Alternatively, if the values are explored simultaneously, the search space of the hyperparameters can be unbearably large;
4. Different stages of the evolutionary process may require different optimal hyperparameter values;
5. Optimal values for a given algorithm tendentiously differ from one problem to another, which implies repeating the tuning for each new problem.

An alternative to static tuning is parameter control, where hyperparameters change dynamically during the evolutionary run [23]. Deterministic control relies on some time-dependent function to update hyperparameters [80], while adaptive control adjusts hyperparameter values dynamically based on information extracted from the evolutionary process. A significant amount of work suggests the superiority of hyperparameter control over tuning [21, 23, 57, 70].

Traditionally, researchers focused on the control of specific hyperparameters related to genetic operators [18, 23, 33, 57] or selection methods [2, 21]. Several works found that population size was one of the most influential control variables in evolutionary search [9, 22, 23, 25] as it can have large impacts on the behavior (final performance, population diversity, search speed, and others) of the optimization process. For traditional GP, considerable research has focused on understanding the impact of population size on its search dynamics and on crafting strategies to adapt or control it [12, 24, 26, 27, 55, 77, 79]. Additionally, multiple studies have explored adaptive probabilities for crossover and mutation [51, 69, 89], as well as broader forms of control including elitism, crossover, mutation, and replacement parameters [78].

Hyperparameter tuning remains a best-practice at time of publication because until recently, (self-)adaptive optimizers have received little research attention in CGP. Although Kalkreuth et al. [47] had some success in 2015, recently, optimizers and especially modern (self-)adaptive versions were not the core focus of the field. However, both in GP and especially the wider EC community, (self-)adaptivity of evolutionary algorithms has shown great advantages over the non-adaptive counterparts [21, 23, 33, 35, 55, 57]. Having (self-)adaptive operators as part of a CGP optimizer could provide two essential benefits. First, the optimization might become more efficient and more effective. Second, self-adaptive operators often reduce the effect of specific hyperparameters on the performance. Ideally, this leads to hyperparameter-free training.

While this can come at a cost regarding the overall computational effort needed for a single training instance, it can lead to less hyperparameter tuning (or better guided tuning) and, crucially, assist those practitioners without in-depth knowledge on beneficial configurations of CGP parameters, which in turn leads to wider adoption of CGP. We thus encourage further study of and a

shift towards self-adaptive methods once the dynamics and effects are better understood.

6.6 Hybridization of Reorder and Subgraph

To this day, *other influences* of positional bias or reordering nodes are *unclear*. In particular, its effect on *crossover* strategies are not fully understood yet, although first steps were accomplished by Cui et al. [14]. Further investigating the effects of reordering nodes with crossover might lead to new insights. Thus, we further propose the investigation of a *hybridization between reordering nodes and the subgraph crossover mechanism* (cf. Sect. 4.1).

Kalkreuth’s [46] recombination method only swaps one subgraph between individuals. As a result, it can also be seen as a “highly guided and specialized one-point crossover” (with one-point crossover showing favorable properties in GP [75]). However, this operator must choose one crossover point within the range of active nodes. Due to positional bias, this crossover point is highly likely to be close to the input nodes—as almost all active nodes cluster near input nodes. This was demonstrated by Kocherovsky et al. in 2025 [54]. By eliminating positional bias, we believe that the crossover point for the subgraph crossover is moved towards the middle of CGP’s graph. This leads to a more uniform distribution of swapped genetic material. In turn, CGP’s genotype changes are more substantial. This increases *neutral genetic drift*, a desirable trait in CGP which leads to performance boosts [39, 84, 90].

7 Conclusion

Cartesian Genetic Programming was proposed nearly thirty years ago, and its study and refinement has progressed significantly since then. However, as with any design paradigm, certain practices and assumptions become frozen accidents.

In this work, we have sought to discuss misconceptions prevalent in the field, particularly around:

1. Crossover, positional bias, and benchmarking;
2. A holistic review successful genetic operators, such as subgraph and node-preserving crossover;
3. Promoting *new* best practices based on the recent literature, such as hyperparameter tuning, cross-domain benchmarking, and certain methods of evaluation;
4. Introducing new directions for research around selection methods, crossover fundamentals, homologous operators, fitness landscapes, adaptive methods, and practice hybridization.

It is our hope that this work contributes to the general field of CGP by encouraging both researchers and general practitioners to approach CGP from new angles in academic, educational, and industrial contexts. It is our position that a fresh look at CGP from new points of view—indebted to older viewpoints, of course—could be vital to overcoming existing issues and discovering both new dynamics to be observed and new problems to be solved, enhancing the abilities robustness not just of CGP models themselves, but the field as a whole.

Acknowledgements. Partial support was received by the Bavarian State Ministry of Science and the Arts through the Augsburg AI Production Network as part of the High-Tech Agenda Plus. This work was supported by an Alexander von Humboldt Professorship in AI awarded to Holger H. Hoos, with funding provided by the German Federal Ministry of Education and Research.

References

1. Adewuya, A.: New methods in genetic search with real-valued chromosomes. Bachelor's thesis, Mississippi State University (1996)
2. Affenzeller, M., Wagner, S.: Offspring selection: a new self-adaptive selection scheme for genetic algorithms. In: Ribeiro, B., Albrecht, R.F., Dobnikar, A., Pearson, D.W., Steele, N.C. (eds.) *Adaptive and Natural Computing Algorithms*, pp. 218–221. Springer, Vienna (2005). https://doi.org/10.1007/3-211-27389-1_52
3. Bakurov, I., Haut, N., Banzhaf, W.: Sharpness-Aware Minimization in Genetic Programming, pp. 151–175. Springer, Singapore (2025). https://doi.org/10.1007/978-981-96-0077-9_8
4. Bakurov, I., Murphy, A., Ofria, C., Banzhaf, W.: A comparison of tournament and lexcase selection paradigms in regression problems: error-based fitness versus correlation fitness. In: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2025*, pp. 970–979. Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3712256.3726448>
5. Banzhaf, W., Bakurov, I.: On the nature of the phenotype in tree genetic programming. In: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2024*, pp. 868–877. Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3638529.3654129>
6. Banzhaf, W., Hu, T., Ochoa, G.: How the combinatorics of neutral spaces leads genetic programming to discover simple solutions. In: Winkler, S., Trujillo, L., Ofria, C., Hu, T. (eds.) *Genetic Programming Theory and Practice XX*, pp. 65–86. Genetic and Evolutionary Computation, Springer, Michigan State University, USA (2023). https://doi.org/10.1007/978-981-99-8413-8_4
7. Banzhaf, W., Leier, A.: Evolution on Neutral Networks in Genetic Programming, pp. 207–221. Springer US, Boston, MA (2006). https://doi.org/10.1007/0-387-28111-8_14
8. Bartz-Beielstein, T., et al.: Benchmarking in optimization: best practice and open issues (2020). <https://arxiv.org/abs/2007.03488>
9. Bingchuan, W., Shui, Z.Y., Feng, Y., Ma, Z.: Evolutionary algorithm with dynamic population size for constrained multiobjective optimization. *Swarm Evol. Comput.* **73**, 101104 (2022). <https://doi.org/10.1016/j.swevo.2022.101104>

10. Blickle, T., Thiele, L.: A mathematical analysis of tournament selection. In: Proceedings of the 6th International Conference on Genetic Algorithms, pp. 9–16. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1995). <https://doi.org/10.5555/645514.658088>
11. Calvo, B., Ceberio, J., Lozano, J.A.: Bayesian inference for algorithm ranking analysis. In: Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO 2018, pp. 324–325. Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3205651.3205658>
12. Castelli, M., Manzoni, L., Silva, S., Vanneschi, L., Popović, A.: The influence of population size in geometric semantic GP. *Swarm Evol. Comput.* **32**, 110–120 (2017). <https://doi.org/10.1016/j.swevo.2016.05.004>
13. Clegg, J., Walker, J.A., Miller, J.F.: A new crossover technique for cartesian genetic programming. In: Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, GECCO 2007, pp. 1580–1587. Association for Computing Machinery, New York, NY, USA (2007). <https://doi.org/10.1145/1276958.1277276>
14. Cui, H., Heider, M., Hähner, J.: Positional bias does not influence cartesian genetic programming with crossover. In: Affenzeller, M., et al. (eds.) *Parallel Problem Solving from Nature – PPSN XVIII*, pp. 151–167. Springer Nature, Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-70055-2_10
15. Cui, H., Margraf, A., Hähner, J.: Analysing the influence of reorder strategies for cartesian genetic programming. *SN Comput. Sci.* **6**(7), 754 (2025). <https://doi.org/10.1007/s42979-025-04296-4>
16. Cui, H., Margraf, A., Heider, M., Hähner, J.: Towards understanding crossover for cartesian genetic programming. In: *Towards Understanding Crossover for Cartesian Genetic Programming*, pp. 308–314 (2023). <https://doi.org/10.5220/0012231400003595>
17. Cui, H., Margraf, A., Hähner, J.: Equidistant reorder operator for cartesian genetic programming. In: van Stein, N., Marcelloni, F., Lam, H.K., Cottrell, M., Filipe, J. (eds.) *Proceedings of the 15th International Joint Conference on Computational Intelligence - ECTA*, 13–15 November 2023, in Rome, Italy, pp. 64 – 74 (2023). <https://doi.org/10.5220/0012174100003595>
18. Das, S., Abraham, A., Chakraborty, U.K., Konar, A.: Differential evolution using a neighborhood-based mutation operator. *IEEE Trans. Evol. Comput.* **13**(3), 526–553 (2009). <https://doi.org/10.1109/TEVC.2008.2009457>
19. De Jong, K.: Parameter setting in EAs: a 30 year perspective, pp. 1–18. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-69432-8_1
20. De La Torre, C., Cussat-Blanc, S., Wilson, D., Lavinas, Y.: On search trajectory networks for graph genetic programming. In: *Proceedings of the Genetic and Evolutionary Computation Conference Companion, GECCO 2024 Companion*, pp. 1681–1685. Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3638530.3664169>
21. Droste, S., Jansen, T., Wegener, I.: Dynamic parameter control in simple evolutionary algorithms. In: Martin, W.N., Spears, W.M. (eds.) *Foundations of Genetic Algorithms*, vol. 6, pp. 275–294. Morgan Kaufmann, San Francisco (2001). <https://doi.org/10.1016/B978-155860734-7/50098-6>. <https://www.sciencedirect.com/science/article/pii/B9781558607347500986>
22. Eiben, A.E., Marchiori, E., Valkó, V.A.: Evolutionary algorithms with on-the-fly population size adjustment. In: Yao, X., et al. (eds.) *Parallel Problem Solving from Nature - PPSN VIII*, pp. 41–50. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30217-9_5

23. Eiben, A., Hinterding, R., Michalewicz, Z.: Parameter control in evolutionary algorithms. *IEEE Trans. Evol. Comput.* **3**(2), 124–141 (1999). <https://doi.org/10.1109/4235.771166>
24. Farinati, D., Bakurov, I., Vanneschi, L.: A study of dynamic populations in geometric semantic genetic programming. *Inf. Sci.* **648**, 119513 (2023). <https://doi.org/10.1016/j.ins.2023.119513>. <https://www.sciencedirect.com/science/article/pii/S0020025523010988>
25. Farinati, D., Vanneschi, L.: A survey on dynamic populations in bio-inspired algorithms. *Genet. Program Evolvable Mach.* **25**(2), 19 (2024). <https://doi.org/10.1007/s10710-024-09492-4>
26. Fernandez, F., Tomassini, M., Vanneschi, L.: Saving computational effort in genetic programming by means of plagues. In: *The 2003 Congress on Evolutionary Computation, CEC 2003*, vol. 3, pp. 2042–2049 (2003). <https://doi.org/10.1109/CEC.2003.1299924>
27. Fernandez, F., Vanneschi, L., Tomassini, M.: The effect of plagues in genetic programming: a study of variable-size populations. In: Ryan, C., Soule, T., Keijzer, M., Tsang, E., Poli, R., Costa, E. (eds.) *Genetic Programming*, pp. 317–326. Springer, Heidelberg (2003). https://doi.org/10.1007/3-540-36599-0_29
28. de Franca, F.O., et al.: SRBench++: principled benchmarking of symbolic regression with domain-expert interpretation. *IEEE Trans. Evol. Comput.* **29**(4), 1127–1137 (2025). <https://doi.org/10.1109/TEVC.2024.3423681>
29. Francone, F.D., Conrads, M., Banzhaf, W., Nordin, P.: Homologous crossover in genetic programming. In: *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation, GECCO'99*, vol. 2, pp. 1021–1026. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1999). <https://doi.org/10.5555/2934046.2934059>
30. Goldman, B.W., Punch, W.F.: Length bias and search limitations in cartesian genetic programming. In: *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation, GECCO 2013*, pp. 933–940. Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2463372.2463482>
31. Goldman, B.W., Punch, W.F.: Reducing wasted evaluations in cartesian genetic programming. In: *Proceedings of the 16th European Conference on Genetic Programming, EuroGP 2013*, pp. 61–72. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-37207-0_6
32. Goldman, B.W., Punch, W.F.: Analysis of cartesian genetic programming's evolutionary mechanisms **19**(3), 359–373 (2015). <https://doi.org/10.1109/TEVC.2014.2324539>
33. Hansen, N., Ostermeier, A.: Completely derandomized self-adaptation in evolution strategies. *Evol. Comput.* **9**(2), 159–195 (2001). <https://doi.org/10.1162/106365601750190398>
34. Haut, N., Banzhaf, W., Punch, B.: Correlation versus RMSE loss functions in symbolic regression tasks. In: Trujillo, L., Winkler, S.M., Silva, S., Banzhaf, W. (eds.) *Genetic Programming Theory and Practice XIX*, pp. 31–55. Springer (2023). https://doi.org/10.1007/978-981-19-8460-0_2
35. Heider, M., Krischan, M., Sraj, R., Hähner, J.: Exploring self-adaptive genetic algorithms to combine compact sets of rules. In: *2024 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1–8 (2024). <https://doi.org/10.1109/CEC60901.2024.10612101>

36. Helmuth, T., Kelly, P.: PSB2: the second program synthesis benchmark suite. In: Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2021, pp. 785–794. Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3449639.3459285>
37. Helmuth, T., Spector, L.: General program synthesis benchmark suite. In: Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation, GECCO 2015, pp. 1039–1046. Association for Computing Machinery, New York, NY, USA (2015). <https://doi.org/10.1145/2739480.2754769>
38. Hu, T., Banzhaf, W.: Neutrality and variability: two sides of evolvability in linear genetic programming. In: Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation, GECCO 2009, pp. 963–970. Association for Computing Machinery, New York, NY, USA (2009). <https://doi.org/10.1145/1569901.1570033>
39. Hu, T., Banzhaf, W.: Neutrality, robustness, and evolvability in genetic programming. In: Riolo, R., Worzel, B., Goldman, B., Tozier, B. (eds.) Genetic Programming Theory and Practice XIV, pp. 101–117. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-97088-2_7
40. Hu, T., Banzhaf, W., Ochoa, G.: How neutrality shapes evolution: simplicity bias and search. In: Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2025, pp. 1008–1016. Association for Computing Machinery, New York, NY, USA (2025). <https://doi.org/10.1145/3712256.3726454>
41. Hunter, N.: Meiotic recombination: the essence of heredity. *Cold Spring Harbor Perspect. Biol.* **7** 12 (2015). <https://doi.org/10.1101/cshperspect.a016618>. <https://api.semanticscholar.org/CorpusID:33542130>
42. Husa, J., Kalkreuth, R.: A comparative study on crossover in cartesian genetic programming. In: Castelli, M., Sekanina, L., Zhang, M., Cagnoni, S., García-Sánchez, P. (eds.) Genetic Programming, pp. 203–219. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-77553-1_13
43. Iqbal, S., Hoque, M.T.: A homologous gene replacement based genetic algorithm. In: Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion, pp. 91–92 (2016)
44. Kalkreuth, R.: A comprehensive study on subgraph crossover in cartesian genetic programming. In: 12th International Joint Conference on Computational Intelligence, pp. 59–70 (2020). <https://doi.org/10.5220/0010110700590070>
45. Kalkreuth, R.: Reconsideration and extension of cartesian genetic programming. Ph.D. thesis, Department of Computer Science, University of Dortmund (2021). <https://doi.org/10.17877/DE290R-22504>
46. Kalkreuth, R., Rudolph, G., Droschinsky, A.: A new subgraph crossover for cartesian genetic programming. In: McDermott, J., Castelli, M., Sekanina, L., Haasdijk, E., García-Sánchez, P. (eds.) European Conference on Genetic Programming, pp. 294–310. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-55696-3_19
47. Kalkreuth, R., Rudolph, G., Krone, J.: Improving convergence in cartesian genetic programming using adaptive crossover, mutation and selection. In: 2015 IEEE Symposium Series on Computational Intelligence, pp. 1415–1422 (2015). <https://doi.org/10.1109/SSCI.2015.201>
48. Kalkreuth, R., Vašíček, Z., Husa, J., Vermetten, D., Ye, F., Bäck, T.: General Boolean function benchmark suite. In: Proceedings of the 17th ACM/SIGEVO Conference on Foundations of Genetic Algorithms, FOGA 2023, pp. 84–95. Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3594805.3607131>

49. Kaufmann, P., Kalkreuth, R.: Parametrizing cartesian genetic programming: an empirical study. In: Kern-Isberner, G., Fürnkranz, J., Thimm, M. (eds.) *KI 2017: Advances in Artificial Intelligence*, pp. 316–322. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-67190-1_26
50. Kaufmann, P., Kalkreuth, R.: On the parameterization of cartesian genetic programming. In: *2020 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1–8 (2020). <https://doi.org/10.1109/CEC48606.2020.9185492>
51. Kefer, K., Affenzeller, M., Winkler, S.: Adaptive operators for genetic programming to identify optimal energy flow controllers. *Procedia Comput. Sci.* **253**, 1991–2002 (2025). <https://doi.org/10.1016/j.procs.2025.01.261>. <https://www.sciencedirect.com/science/article/pii/S1877050925002698>. 6th International Conference on Industry 4.0 and Smart Manufacturing
52. Kocherovsky, M., Bakurov, I., Banzhaf, W.: Node preservation and its effect on crossover in cartesian genetic programming (2025). <https://arxiv.org/abs/2511.00634>
53. Kocherovsky, M., Banzhaf, W.: Crossover destructiveness in cartesian versus linear genetic programming. In: *ALIFE 2024: Proceedings of the 2024 Artificial Life Conference*, p. 20. Artificial Life Conference Proceedings, International Society for Artificial Life (2024). https://doi.org/10.1162/isal_a_00735
54. Kocherovsky, M., Kianinejad, M., Bakurov, I., Banzhaf, W.: On the effectiveness of crossover operators in cartesian genetic programming. In: Xue, B., Manzoni, L., Bakurov, I. (eds.) *Genetic Programming*, pp. 68–84. Springer Nature Switzerland, Cham (2025). https://doi.org/10.1007/978-3-031-89991-1_5
55. Kouchakpour, P., Zaknich, A., Braunl, T.: Dynamic population variation in genetic programming. *Inf. Sci.* **179**, 1078–1091 (2009). <https://doi.org/10.1016/j.ins.2008.12.009>
56. Kruschke, J.K.: Bayesian estimation supersedes the T test. *J. Exp. Psychol. Gen.* **142**(2), 573 (2013). <https://doi.org/10.1037/a0029146>
57. Lima, C.F., Lobo, F.G.: Parameter-less optimization with the extended compact genetic algorithm and iterated local search. In: Deb, K. (ed.) *Genetic and Evolutionary Computation – GECCO 2004*, pp. 1328–1339. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-24854-5_127
58. López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., Stützle, T.: The irace package: iterated racing for automatic algorithm configuration. *Oper. Res. Perspect.* **3**, 43–58 (2016). <https://doi.org/10.1016/j.orp.2016.09.002>. <https://www.sciencedirect.com/science/article/pii/S2214716015300270>
59. Malan, K.M., Engelbrecht, A.P.: Quantifying ruggedness of continuous landscapes using entropy. In: *2009 IEEE Congress on Evolutionary Computation*, pp. 1440–1447 (2009). <https://doi.org/10.1109/CEC.2009.4983112>
60. Malan, K.M., Engelbrecht, A.P.: A survey of techniques for characterising fitness landscapes and some possible ways forward. *Inf. Sci.* **241**, 148–163 (2013). <https://doi.org/10.1016/j.ins.2013.04.015>. <https://www.sciencedirect.com/science/article/pii/S0020025513003125>
61. McDermott, J., et al.: Genetic programming benchmarks: looking back and looking forward. *SIGEVolution* **15**(3) (2022). <https://doi.org/10.1145/3578482.3578483>
62. Miller, J.F.: An empirical study of the efficiency of learning Boolean functions using a cartesian genetic programming approach. In: *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation, GECCO 1999*, vol. 2, pp. 1135–1142. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1999). <https://doi.org/10.5555/2934046.2934074>. <https://dl.acm.org/doi/10.5555/2934046.2934074>

63. Miller, J.F., Smith, S.L.: Redundancy and computational efficiency in cartesian genetic programming. *IEEE Trans. Evol. Comput.* **10**(2), 167–174 (2006). <https://doi.org/10.1109/TEVC.2006.871253>
64. Miller, J.F., Harding, S.L.: Cartesian genetic programming. In: *Proceedings of the 10th Annual Conference Companion on Genetic and Evolutionary Computation, GECCO 2008*, pp. 2701–2726. Association for Computing Machinery, New York, NY, USA (2008). <https://doi.org/10.1145/1388969.1389075>
65. Mühlenbein, H., Schomisch, M., Born, J.: The parallel genetic algorithm as function optimizer. *Parallel Comput.* **17**, 619–632 (1991). [https://doi.org/10.1016/S0167-8191\(05\)80052-3](https://doi.org/10.1016/S0167-8191(05)80052-3). <https://api.semanticscholar.org/CorpusID:2485505>
66. Nadizar, G., Rusin, F., Medvet, E., Ochoa, G.: The role of stepping stones in map-elites: insights from search trajectory networks. In: *Genetic Programming: 28th European Conference, EuroGP 2025, Held as Part of EvoStar 2025, Trieste, Italy, 23–25 April 2025, Proceedings*, pp. 224–239. Springer, Heidelberg (2025). https://doi.org/10.1007/978-3-031-89991-1_14
67. Ochoa, G., Malan, K.M., Blum, C.: Search trajectory networks of population-based algorithms in continuous spaces. In: *Applications of Evolutionary Computation: 23rd European Conference, EvoApplications 2020, Held as Part of EvoStar 2020, Seville, Spain, 15–17 April 2020, Proceedings*, pp. 70–85. Springer, Heidelberg (2020). https://doi.org/10.1007/978-3-030-43722-0_5
68. Ochoa, G., Malan, K.M., Blum, C.: Search trajectory networks: a tool for analysing and visualising the behaviour of metaheuristics. *Appl. Soft Comput.* **109**, 107492 (2021). <https://doi.org/10.1016/j.asoc.2021.107492>. <https://www.sciencedirect.com/science/article/pii/S1568494621004154>
69. Oh, S., Suh, W.H., Ahn, C.W.: Self-adaptive genetic programming for manufacturing big data analysis. *Symmetry* **13**(4) (2021). <https://doi.org/10.3390/sym13040709>. <https://www.mdpi.com/2073-8994/13/4/709>
70. Ojha, V., Timmis, J., Nicosia, G.: Assessing ranking and effectiveness of evolutionary algorithm hyperparameters using global sensitivity analysis methodologies. *Swarm Evol. Comput.* **74**, 101130 (2022). <https://doi.org/10.1016/j.swevo.2022.101130>. <https://www.sciencedirect.com/science/article/pii/S2210650222001006>
71. Olson, R.S., La Cava, W., Orzechowski, P., Urbanowicz, R.J., Moore, J.H.: PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData Mining* **10**(1), 36 (2017). <https://doi.org/10.1186/s13040-017-0154-4>
72. Orzechowski, P., La Cava, W., Moore, J.H.: Where are we now? A large benchmark study of recent symbolic regression methods. In: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2018*, pp. 1183–1190. Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3205455.3205539>
73. Payne, A.J., Stepney, S.: Representation and structural biases in CGP. In: *2009 IEEE Congress on Evolutionary Computation*, pp. 1064–1071 (2009). <https://doi.org/10.1109/CEC.2009.4983064>
74. Platel, M.D., Clergue, M., Collard, P.: Maximum homologous crossover for linear genetic programming. In: *Proceedings of the 6th European Conference on Genetic Programming, EuroGP 2003*, pp. 194–203. Springer, Heidelberg (2003). <https://doi.org/10.5555/1762668.1762687>
75. Poli, R., Langdon, W.B.: Genetic programming with one-point crossover. In: Chawdhry, P.K., Roy, R., Pant, R.K. (eds.) *Soft Computing in Engineering Design and Manufacturing*, pp. 180–189. Springer, London (1998). https://doi.org/10.1007/978-1-4471-0427-8_20

76. Poli, R., Langdon, W.B.: Schema theory for genetic programming with one-point crossover and point mutation. *Evol. Comput.* **6**(3), 231–252 (1998). <https://doi.org/10.1162/evco.1998.6.3.231>
77. Poli, R., McPhee, N.F., Vanneschi, L.: The impact of population size on code growth in GP: analysis and empirical validation. In: *Proceedings of the 10th Annual Conference on Genetic and Evolutionary Computation, GECCO 2008*, pp. 1275–1282. Association for Computing Machinery, New York, NY, USA (2008). <https://doi.org/10.1145/1389095.1389341>
78. Riekert, M., Malan, K.M., Engelbrecht, A.P.: Adaptive genetic programming for dynamic classification problems. In: *Proceedings of the Eleventh Conference on Congress on Evolutionary Computation, CEC 2009*, pp. 674–681. IEEE Press (2009). <https://doi.org/10.1109/CEC.2009.4983010>
79. Rochat, D., Tomassini, M., Vanneschi, L.: Dynamic size populations in distributed genetic programming. In: Keijzer, M., Tettamanzi, A., Collet, P., van Hemert, J., Tomassini, M. (eds.) *Genetic Programming*, pp. 50–61. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-31989-4_5
80. Tanabe, R., Fukunaga, A.S.: Improving the search performance of shade using linear population size reduction. In: *2014 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1658–1665 (2014). <https://doi.org/10.1109/CEC.2014.6900380>
81. Torabi, A., Sharifi, A., Teshnehlab, M.: Using cartesian genetic programming approach with new crossover technique to design convolutional neural networks. *Neural Process. Lett.* **55**(5), 5451–5471 (2023). <https://doi.org/10.1007/s11063-022-11093-0>
82. Trautwein, J., Heider, M., Cui, H., Hähner, J.: Ant-based metaheuristics struggle to solve the cartesian genetic programming learning task. In: Xue, B., Manzoni, L., Bakurov, I. (eds.) *Genetic Programming*, pp. 139–155. Springer Nature Switzerland, Cham (2025)
83. Turner, A.J.: Improving crossover techniques in a genetic program. Master’s thesis, Department of Electronics, University of York (2012)
84. Turner, A.J., Miller, J.F.: Neutral genetic drift: an investigation using cartesian genetic programming **16**(4), 531–558 (2015). <https://doi.org/10.1007/s10710-015-9244-6>
85. Vanneschi, L.: Fitness landscapes and problem hardness in genetic programming. In: *Proceedings of the 12th Annual Conference Companion on Genetic and Evolutionary Computation, GECCO 2010*, pp. 2711–2738. Association for Computing Machinery, New York, NY, USA (2010). <https://doi.org/10.1145/1830761.1830916>
86. Vašíček, Z.: Bridging the gap between evolvable hardware and industry using cartesian genetic programming. In: *Emergence, Complexity and Computation*, vol. 28, pp. 39–55. Springer International Publishing (2018). https://doi.org/10.1007/978-3-319-67997-6_2
87. White, D., et al.: Better GP benchmarks: community survey results and proposals **14**, 3–29 (2013)
88. Wilson, G., Banzhaf, W.: A comparison of cartesian genetic programming and linear genetic programming. In: O’Neill, M., et al. (eds.) *Genetic Programming*, pp. 182–193. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78671-9_16

89. Yin, Z., Brabazon, A., O'Sullivan, C.: Adaptive genetic programming for option pricing. In: Proceedings of the 9th Annual Conference Companion on Genetic and Evolutionary Computation, GECCO 2007, pp. 2588–2594. Association for Computing Machinery, New York, NY, USA (2007). <https://doi.org/10.1145/1274000.1274029>
90. Yu, T., Miller, J.: Finding needles in haystacks is not hard with neutrality. In: Foster, J.A., Lutton, E., Miller, J., Ryan, C., Tettamanzi, A. (eds.) Genetic Programming, pp. 13–25. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45984-7_2